

EECS 348 Assignment 2 — GenAI Code Comparison

- **Author:** Francis Tryban — Student ID 3222774
- **Course:** EECS 348, Software Engineering I, Fall 2026
- **Assignment:** No. 2 — CEO email prioritization via a from-scratch MaxHeap
- **Started:** September 14, 2026
- **Due:** September 17, 2026, 11:59 PM
- **Collaborators / other sources:** (Claude Opus 5) assisted in the gathering of information and previdous drafts of this analysis.

Which two GenAIs, and in what configuration

Claude **Opus 5 Ultra Code** versus **5.6 Sol Ultra**, each driven through its own agentic command-line harness, each with **full computer access enabled**.

Why.

1. *Contemporaneous release.* The two models shipped at roughly the same time, so neither benefits from an extra generation of training. A comparison between a current flagship and a year-old model measures the calendar.
2. *Both are top-of-the-line.* The interesting question is not whether a weak model fails — it is whether two of the strongest available systems, given the same specification and nothing else, converge on the same program. If they diverge here, the divergence is about how each system reads an underspecified spec, which is the real subject of this course.
3. *Working in a harness.* The AIs are only able to see the files wrapped around them. With the split of AGENT.md from PROMPT.md, I will be able to inform the model on the way we do things in AGENT.md while PROMPT.md answers *what* to do.

Method (fixed before any code was generated)

The comparison is only meaningful if the two GenAIs receive identical input. I built identical harnesses around the AIs.

Claude Rendered this visual representation of the file structure:

```
3222774_Tryban_Assignment_2/
├── Claude/
│   ├── AGENT.md          (empty at run 00)
│   └── PROMPT.md         (assignment program spec – byte-identical to
Codex/PROMPT.md)
├── iterations/           (00-raw.c lands here, then 01-*.c, ...)
└── Codex/
    └── AGENT.md          (empty at run 00)
```

```
└─ PROMPT.md      (byte-identical to Claude/PROMPT.md)
└─ iterations/
```

PROMPT.md contains the program specification only — the MaxHeap requirements, the command table, the sample test file, and the byte-exact expected output. It excludes the submission logistics, the rubric, and the analysis instructions. Both files were written from the same source and verified identical.

#PROMPT.md

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

Test File format:

```
| Command | Description |
|---|---|
| `EMAIL <sender category>,<subject line>,<date>` | The emails in the CEO's
Inbox should be placed in queue based on their sender category and date.
The sender categories and priority to be read are as follows: Boss – read
first, Subordinate – read next, Peer – read next, ImportantPerson – read
next, OtherPerson – read last. If there is more than one from a sender,
then the newest email (not the oldest) should be read first. I discovered
this trick while a manager at Sprint. EMAIL is followed by space. The rest
of the fields are delimited. Assume `<sender category>` is one of the five
strings listed above. Assume `<subject line>` is a string which may contain
spaces, but not commas. Assume `<date>` is in the format: MM-DD-YYYY |
| `NEXT` | Next email for the CEO to read. Display the information on the
terminal in the following format: Sender: / Subject: / Date: |
| `READ` | CEO has read email and dealt with it |
| `COUNT` | Display current count of unread emails |
```

The grader will test your code with a different test file. The test file will

begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted

as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
```

```
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Your program should display this:

```
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.
```

Input clarification

The program's expected input is a text file containing one command per line,
such as:

```
```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL Boss,Important,12-20-2024
COUNT
NEXT
READ
```
```

The program must accept that file in either of two ways:

- If a filename is given as the first command-line argument, read the commands
from that file: `./program testfile.txt`
- If no argument is given, read the commands from standard input:
`./program < testfile.txt`

All output goes to standard output. The program exits with status 0 when it completes normally.

AGENT.md is **empty** for run 00. That is the point of run 00: it measures what each system produces from the specification alone, with no coaching. (*Rules will enter AGENT.md after the raw results have been analyzed*)

With full computer access enabled, each system can compile its own code, run it against the sample test file printed in **PROMPT.md**, see its own failures, and fix them before saving. That means **00-raw.c** is **not** a single forward pass from the model. It is the output of an agentic loop: model plus harness plus compiler plus filesystem. **However**, the thought is, while for run 00: full computer access remains enabled at the harness level, the run instruction **forbids** using it to compile, execute, or test. Run 00 is therefore much closer to a comparison of the two **models** than of the two systems. That is the thought.

Turns out both of them will break the rules happily to try and get better result

Run instruction:

Read AGENT.md and PROMPT.md. Write a C program that satisfies PROMPT.md. Save it to iterations/00-raw.c. Do not create or modify any other files.

The sentence "Do not compile, run, or test anything" is absent. Both systems then took the liberty to execute, and both did: Claude compiled with **-Wall -Wextra -pedantic -Werror -fsanitize=address,undefined**, built sample and edge-case inputs, diffed against the expected output, and ran a leak check; Codex compiled with **-Wall -Wextra -Wpedantic -Werror**, piped the sample and edge cases through the binary, and deleted its temporary executable afterward.

Here are the results:

Program A — Claude Opus 5, **Claude/iterations/00-raw.c**

```
/*
 * CEO Email Prioritizer
 *
 * Reads commands (EMAIL, NEXT, READ, COUNT) from a file given as the first
 * command-line argument, or from standard input if no argument is given.
 *
 * Emails are stored in a MaxHeap implemented from scratch on top of a
 * dynamically growing array (list-based heap). The heap acts as a priority
 * queue ordered by:
 * 1. Sender category: Boss > Subordinate > Peer > ImportantPerson >
OtherPerson
 * 2. Date: newer emails before older emails
 * 3. Arrival order: for identical category and date, the later-arriving
 *    email is treated as newer
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <ctype.h>

/* -----
- */
/* Email record
*/
/* -----
- */

typedef struct {
    char *sender;        /* sender category text, e.g. "Boss" */
    char *subject;       /* subject line (may contain spaces) */
    char *date;          /* date text as given, MM-DD-YYYY */
    int  priority;       /* numeric rank of sender category, higher = sooner
*/
    long  dateKey;       /* YYYYMMDD, so larger means newer */
    unsigned long seq;   /* arrival order, used as a final tie-breaker */
} Email;

static char *dupString(const char *s)
{
    size_t len = strlen(s);
    char *copy = malloc(len + 1);
    if (copy == NULL) {
        fprintf(stderr, "Error: out of memory\n");
        exit(1);
    }
    memcpy(copy, s, len + 1);
    return copy;
}

static void freeEmail(Email *e)
{
    free(e->sender);
    free(e->subject);
    free(e->date);
    e->sender = NULL;
    e->subject = NULL;
    e->date = NULL;
}

/* Map a sender category to its priority. Unknown categories rank lowest.
*/
static int categoryPriority(const char *category)
{
    if (strcmp(category, "Boss") == 0)        return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0)        return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0;
}

/* Convert MM-DD-YYYY into a sortable YYYYMMDD number. Returns 0 if
```

```

malformed. */
static long dateToKey(const char *date)
{
    int month, day, year;
    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0;
    }
    return (long)year * 10000L + (long)month * 100L + (long)day;
}

/* Returns nonzero if email a should be read before email b. */
static int higherPriority(const Email *a, const Email *b)
{
    if (a->priority != b->priority) {
        return a->priority > b->priority;
    }
    if (a->dateKey != b->dateKey) {
        return a->dateKey > b->dateKey;
    }
    return a->seq > b->seq;
}

/* -----
- */
/* MaxHeap (list-based)
*/
/* -----
- */

typedef struct {
    Email *items;          /* array storage; children of i are 2i+1 and 2i+2
*/
    size_t size;
    size_t capacity;
} MaxHeap;

static void heapInit(MaxHeap *heap)
{
    heap->items = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

static void heapFree(MaxHeap *heap)
{
    size_t i;
    for (i = 0; i < heap->size; i++) {
        freeEmail(&heap->items[i]);
    }
    free(heap->items);
    heap->items = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

```

```
static size_t heapSize(const MaxHeap *heap)
{
    return heap->size;
}

static int heapIsEmpty(const MaxHeap *heap)
{
    return heap->size == 0;
}

static void swapEmails(Email *a, Email *b)
{
    Email tmp = *a;
    *a = *b;
    *b = tmp;
}

static void siftUp(MaxHeap *heap, size_t index)
{
    while (index > 0) {
        size_t parent = (index - 1) / 2;
        if (!higherPriority(&heap->items[index], &heap->items[parent])) {
            break;
        }
        swapEmails(&heap->items[index], &heap->items[parent]);
        index = parent;
    }
}

static void siftDown(MaxHeap *heap, size_t index)
{
    for (;;) {
        size_t left = 2 * index + 1;
        size_t right = left + 1;
        size_t largest = index;

        if (left < heap->size &&
            higherPriority(&heap->items[left], &heap->items[largest])) {
            largest = left;
        }
        if (right < heap->size &&
            higherPriority(&heap->items[right], &heap->items[largest])) {
            largest = right;
        }
        if (largest == index) {
            break;
        }
        swapEmails(&heap->items[index], &heap->items[largest]);
        index = largest;
    }
}

/* Insert an email; the heap takes ownership of its strings. */
```

```

static void heapPush(MaxHeap *heap, Email email)
{
    if (heap->size == heap->capacity) {
        size_t newCapacity = (heap->capacity == 0) ? 16 : heap->capacity *
2;
        Email *grown = realloc(heap->items, newCapacity * sizeof(Email));
        if (grown == NULL) {
            fprintf(stderr, "Error: out of memory\n");
            exit(1);
        }
        heap->items = grown;
        heap->capacity = newCapacity;
    }
    heap->items[heap->size] = email;
    siftUp(heap, heap->size);
    heap->size++;
}

/* Returns the highest-priority email without removing it, or NULL if
empty. */
static const Email *heapPeek(const MaxHeap *heap)
{
    if (heapIsEmpty(heap)) {
        return NULL;
    }
    return &heap->items[0];
}

/* Removes and frees the highest-priority email. Returns 0 if heap was
empty. */
static int heapPop(MaxHeap *heap)
{
    if (heapIsEmpty(heap)) {
        return 0;
    }
    freeEmail(&heap->items[0]);
    heap->size--;
    if (heap->size > 0) {
        heap->items[0] = heap->items[heap->size];
        siftDown(heap, 0);
    }
    return 1;
}

/* -----
- */
/* Input handling
*/
/* -----
- */

/*
* Reads one line of arbitrary length. Returns a malloc'd string without
the

```

```
* trailing newline, or NULL at end of input.
*/
static char *readLine(FILE *in)
{
    size_t capacity = 128;
    size_t length = 0;
    char *buffer = malloc(capacity);
    int ch;

    if (buffer == NULL) {
        fprintf(stderr, "Error: out of memory\n");
        exit(1);
    }

    while ((ch = fgetc(in)) != EOF) {
        if (ch == '\n') {
            break;
        }
        if (length + 1 >= capacity) {
            char *grown;
            capacity *= 2;
            grown = realloc(buffer, capacity);
            if (grown == NULL) {
                free(buffer);
                fprintf(stderr, "Error: out of memory\n");
                exit(1);
            }
            buffer = grown;
        }
        buffer[length++] = (char)ch;
    }

    if (ch == EOF && length == 0) {
        free(buffer);
        return NULL;
    }

    buffer[length] = '\0';
    return buffer;
}

/* Trims leading and trailing whitespace (including '\r') in place. */
static char *trim(char *s)
{
    char *end;

    while (*s != '\0' && isspace((unsigned char)*s)) {
        s++;
    }
    if (*s == '\0') {
        return s;
    }
    end = s + strlen(s) - 1;
    while (end > s && isspace((unsigned char)*end)) {
```

```
        *end-- = '\\0';
    }
    return s;
}

/* -----
- */
/* Commands
*/
/* -----
- */

/* Parses "<sender category>,<subject line>,<date>" and inserts the email.
*/
static void handleEmail(MaxHeap *heap, char *fields, unsigned long seq)
{
    char *firstComma = strchr(fields, ',');
    char *lastComma = strrchr(fields, ',');
    char *sender;
    char *subject;
    char *date;
    Email email;

    if (firstComma == NULL || lastComma == firstComma) {
        return; /* malformed line; ignore */
    }

    *firstComma = '\\0';
    *lastComma = '\\0';

    sender = trim(fields);
    subject = trim(firstComma + 1);
    date = trim(lastComma + 1);

    email.sender = dupString(sender);
    email.subject = dupString(subject);
    email.date = dupString(date);
    email.priority = categoryPriority(email.sender);
    email.dateKey = dateToKey(email.date);
    email.seq = seq;

    heapPush(heap, email);
}

static void handleNext(const MaxHeap *heap)
{
    const Email *top = heapPeek(heap);
    if (top == NULL) {
        printf("No emails to read.\n");
        return;
    }
    printf("Next email:\n");
    printf("Sender: %s\n", top->sender);
    printf("Subject: %s\n", top->subject);
}
```

```
    printf("Date: %s\n", top->date);
}

static void handleRead(MaxHeap *heap)
{
    /* Reading with an empty inbox is a no-op. */
    heapPop(heap);
}

static void handleCount(const MaxHeap *heap)
{
    printf("There are %lu emails to read.\n", (unsigned
long)heapSize(heap));
}

static void processInput(FILE *in, MaxHeap *heap)
{
    char *raw;
    unsigned long seq = 0;

    while ((raw = readLine(in)) != NULL) {
        char *line = trim(raw);

        if (strncmp(line, "EMAIL", 5) == 0 &&
            (line[5] == ' ' || line[5] == '\t')) {
            handleEmail(heap, line + 6, seq++);
        } else if (strcmp(line, "NEXT") == 0) {
            handleNext(heap);
        } else if (strcmp(line, "READ") == 0) {
            handleRead(heap);
        } else if (strcmp(line, "COUNT") == 0) {
            handleCount(heap);
        }
        /* Blank or unrecognized lines are ignored. */

        free(raw);
    }
}

int main(int argc, char *argv[])
{
    FILE *in = stdin;
    MaxHeap heap;

    if (argc > 1) {
        in = fopen(argv[1], "r");
        if (in == NULL) {
            fprintf(stderr, "Error: could not open file '%s'\n", argv[1]);
            return 1;
        }
    }

    heapInit(&heap);
    processInput(in, &heap);
}
```

```
    heapFree(&heap);

    if (in != stdin) {
        fclose(in);
    }

    return 0;
}
```

Program B — Codex gpt-5.6-sol, Codex/iterations/00-raw.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *sender;
    char *subject;
    char *date;
    int sender_priority;
    int date_value;
    unsigned long insertion_order;
} Email;

typedef struct {
    Email *items;
    size_t size;
    size_t capacity;
} MaxHeap;

static void allocation_error(void)
{
    fprintf(stderr, "Unable to allocate memory.\n");
    exit(EXIT_FAILURE);
}

static char *copy_string(const char *text)
{
    size_t length = strlen(text) + 1;
    char *copy = malloc(length);

    if (copy == NULL) {
        allocation_error();
    }
    memcpy(copy, text, length);
    return copy;
}

static int category_priority(const char *sender)
{
    if (strcmp(sender, "Boss") == 0) {
```

```
        return 5;
    }
    if (strcmp(sender, "Subordinate") == 0) {
        return 4;
    }
    if (strcmp(sender, "Peer") == 0) {
        return 3;
    }
    if (strcmp(sender, "ImportantPerson") == 0) {
        return 2;
    }
    return 1;
}

static int date_priority(const char *date)
{
    int month;
    int day;
    int year;

    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0;
    }
    return year * 10000 + month * 100 + day;
}

static int is_higher_priority(const Email *left, const Email *right)
{
    if (left->sender_priority != right->sender_priority) {
        return left->sender_priority > right->sender_priority;
    }
    if (left->date_value != right->date_value) {
        return left->date_value > right->date_value;
    }
    return left->insertion_order < right->insertion_order;
}

static void swap_emails(Email *left, Email *right)
{
    Email temporary = *left;
    *left = *right;
    *right = temporary;
}

static void heap_init(MaxHeap *heap)
{
    heap->items = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

static void heap_insert(MaxHeap *heap, Email email)
{
    size_t index;
```

```
if (heap->size == heap->capacity) {
    size_t new_capacity = heap->capacity == 0 ? 8 : heap->capacity * 2;
    Email *new_items = realloc(heap->items, new_capacity *
sizeof(*new_items));

    if (new_items == NULL) {
        allocation_error();
    }
    heap->items = new_items;
    heap->capacity = new_capacity;
}

index = heap->size;
heap->items[index] = email;
heap->size++;

while (index > 0) {
    size_t parent = (index - 1) / 2;

    if (!is_higher_priority(&heap->items[index], &heap->items[parent]))
    {
        break;
    }
    swap_emails(&heap->items[index], &heap->items[parent]);
    index = parent;
}

static const Email *heap_max(const MaxHeap *heap)
{
    if (heap->size == 0) {
        return NULL;
    }
    return &heap->items[0];
}

static void free_email(Email *email)
{
    free(email->sender);
    free(email->subject);
    free(email->date);
}

static void heap_remove_max(MaxHeap *heap)
{
    size_t index = 0;

    if (heap->size == 0) {
        return;
    }

    free_email(&heap->items[0]);
    heap->size--;
```

```
if (heap->size == 0) {
    return;
}

heap->items[0] = heap->items[heap->size];

for (;;) {
    size_t left = index * 2 + 1;
    size_t right = left + 1;
    size_t largest = index;

    if (left < heap->size &&
        is_higher_priority(&heap->items[left], &heap->items[largest]))
    {
        largest = left;
    }
    if (right < heap->size &&
        is_higher_priority(&heap->items[right], &heap->items[largest]))
    {
        largest = right;
    }
    if (largest == index) {
        break;
    }
    swap_emails(&heap->items[index], &heap->items[largest]);
    index = largest;
}

static void heap_destroy(MaxHeap *heap)
{
    size_t index;

    for (index = 0; index < heap->size; index++) {
        free_email(&heap->items[index]);
    }
    free(heap->items);
}

static char *read_line(FILE *input)
{
    size_t length = 0;
    size_t capacity = 128;
    char *line = malloc(capacity);
    int character;

    if (line == NULL) {
        allocation_error();
    }

    while ((character = fgetc(input)) != EOF && character != '\n') {
        if (length + 1 >= capacity) {
            char *larger_line;
```

```
        capacity *= 2;
        larger_line = realloc(line, capacity);
        if (larger_line == NULL) {
            free(line);
            allocation_error();
        }
        line = larger_line;
    }
    line[length++] = (char) character;
}

if (character == EOF && length == 0) {
    free(line);
    return NULL;
}
if (length > 0 && line[length - 1] == '\r') {
    length--;
}
line[length] = '\0';
return line;
}

static void add_email_command(MaxHeap *heap, char *fields,
                             unsigned long insertion_order)
{
    char *first_comma = strchr(fields, ',');
    char *second_comma;
    Email email;

    if (first_comma == NULL) {
        return;
    }
    second_comma = strchr(first_comma + 1, ',');
    if (second_comma == NULL) {
        return;
    }

    *first_comma = '\0';
    *second_comma = '\0';

    email.sender = copy_string(fields);
    email.subject = copy_string(first_comma + 1);
    email.date = copy_string(second_comma + 1);
    email.sender_priority = category_priority(email.sender);
    email.date_value = date_priority(email.date);
    email.insertion_order = insertion_order;
    heap_insert(heap, email);
}

int main(int argc, char *argv[])
{
    FILE *input = stdin;
    MaxHeap heap;
    char *line;
```

```

unsigned long insertion_order = 0;

if (argc > 2) {
    fprintf(stderr, "Usage: %s [input_file]\n", argv[0]);
    return EXIT_FAILURE;
}
if (argc == 2) {
    input = fopen(argv[1], "r");
    if (input == NULL) {
        fprintf(stderr, "Unable to open input file: %s\n", argv[1]);
        return EXIT_FAILURE;
    }
}

heap_init(&heap);
while ((line = read_line(input)) != NULL) {
    if (strncmp(line, "EMAIL ", 6) == 0) {
        add_email_command(&heap, line + 6, insertion_order++);
    } else if (strcmp(line, "NEXT") == 0) {
        const Email *email = heap_max(&heap);

        if (email != NULL) {
            printf("Next email:\n");
            printf("Sender: %s\n", email->sender);
            printf("Subject: %s\n", email->subject);
            printf("Date: %s\n", email->date);
        }
    } else if (strcmp(line, "READ") == 0) {
        heap_remove_max(&heap);
    } else if (strcmp(line, "COUNT") == 0) {
        printf("There are %zu emails to read.\n", heap.size);
    }
    free(line);
}

heap_destroy(&heap);
if (input != stdin) {
    fclose(input);
}
return 0;
}

```

2e. Analysis: correctness, execution time, space complexity, maintainability

Before running anything: Claude's file is longer, 382 lines against 293, split into more small functions, and has real comments. Codex's is shorter and has almost none. Both build the queue the same way and both handle dates the same way.

Everything is measured, not judged by eye. The script is [evidence/run-tests.sh](#) and its full output is [evidence/run00-bench-local.txt](#).

Test execution — run the code to verify it actually works. Both compile with the course command and both reproduce the assignment's sample output exactly. Not "looks right" — I had the computer compare character by character.

I also tested both ways of supplying the test file, named on the command line and piped in, and both programs handle both. Now this is to be expected. The sample and its expected output are printed inside `PROMPT.md`, and both models were able to compile and run while writing, so both tested against that sample themselves. It was never going to separate them.

So I tested something they could not have rehearsed. I generated 500 random emails, had each program read them all and hand them back in priority order, and compared that order against a sorting program I wrote separately in Python. Both got all 500 right.

2e. Analysis: correctness, execution time, space complexity, maintainability

Everything is measured, not judged by eye. I had Claude write a script to gather facts on the two programs. script is `evidence/run-tests.sh` and its full output is `evidence/run00-bench-local.txt`.

What I could tell just by reading the two files

Claude's is 382 lines across 23 functions with 53 comment lines. Codex's is 293 lines across 14 functions with 4. Both build the queue the same way and both handle dates the same way. That is all I could tell, and none of it answers which one is better. Everything below is me going and getting facts.

Correctness

does it run, does it handle edge cases, and does it handle errors properly.

Test execution — run the code to verify it actually works. Both compile and both reproduce the assignment's sample output exactly. Not "looks right" — I had the computer compare character by character. I also tested both ways of supplying the test file, named on the command line and piped in, and both programs handle both.

That result is weaker than it looks. The sample and its expected output are printed inside `PROMPT.md`, and both models were able to compile and run while writing, so both tested against that sample themselves.

So I tested something they could not have rehearsed. I generated 500 random emails, had each program read them all and hand them back in priority order, and compared that order against a sorting program I wrote separately in Python. Both got all 500 right.

Edge cases — unusual inputs, empty values, boundary conditions

The assignment names several of these and never shows what the output should be. This is where the two programs stop agreeing:

Edge case	Claude	Codex
Ask for the next email when the inbox is empty	prints No emails to read.	prints nothing
Delete when the inbox is empty	nothing, exits normally	nothing, exits normally
Ask twice with no delete between	same answer both times	same answer both times
Delete twice with no ask between	both gone silently	both gone silently
Count at zero and at one	There are 0/1 emails to read.	identical
Two emails, same sender type, same date	the later one in the file wins	the earlier one wins
A sender type outside the five listed	kept, sorted last	kept, sorted last
An EMAIL line missing its commas	skipped	skipped
Blank lines	ignored	ignored
Empty file	exits normally	exits normally

Ten cases, two real disagreements. Neither program crashed, hung, or produced garbage on any input I tried, so both clear the boundary-condition bar. They simply made different choices where the assignment was silent.

Error handling — meaningful error messages instead of silent failures or crashing.

Situation	Claude	Codex	Which matches the lecture
Next email requested, inbox empty	No emails to read.	prints nothing	Claude — Codex fails silently
Extra argument after the filename	ignores it, exits 0	Usage: ./codex [input_file] , exits 1	Codex — Claude fails silently
File that cannot be opened	Error: could not open file 'x'	Unable to open input file: x	tie, both meaningful
Out of memory	message, exits 1	message, exits 1	tie

One silent failure each. Codex's is the more expensive one, because the empty inbox is a case the assignment promises will be tested.

Neither program crashes on any input tested, which is worth stating plainly.

Time complexity

Big-O gives you an idea, but **execution times are what is important**.

Static code analysis - Both programs use the same structure. Adding an email walks up the tree, removing one walks down it, and the number of steps is the height of the tree, so each command costs about $\log n$ steps for n emails. Reading n emails and then reading them all back out is therefore about $n \log n$ steps in both. Neither program recurses, so there is no hidden cost there. Neither calls a standard library function inside a loop in a way that inflates the cost.

By static analysis alone these two are the same program wearing different names.

Empirical benchmarking - Measure execution time across scaling input sizes and plot the growth curve.** Each figure is 100 runs, user CPU only:

Emails (n)	Claude	Codex
100	0.147 s	0.137 s
1,000	0.233 s	0.212 s
10,000	1.269 s	1.226 s

The curve is the answer to the Big-O question: ten times the emails costs roughly ten times the time, so the growth is linear-ish, not quadratic. That matches what static analysis predicted.

Codex is about 4% faster at every size. I pushed to half a million emails to see whether that held, and it did not — at 500,000 the two come out at Codex 0.687 s and Claude 0.672 s and the sign of the gap flips between runs. A 4% gap that disappears under a bigger test is measurement noise, most of it the cost of starting the program 100 times rather than anything either program does. **There is no execution-time difference between these two programs.**

LLM-as-a-judge and prompt analysis.

I did use this, in a form the lecture does not describe: after each model saved its program, I asked it in the same session to write down every place the assignment was silent, what it chose, and what it rejected. Those two files are [iterations/00-report.md](#). That is where the disagreement table above came from, and it is far cheaper than inferring the same list by reading both programs. I treated the reports as claims and checked them against the programs' actual behavior rather than trusting them.

Here is a table of the combined results:

Generation cost — measured, not self-reported

Source: each CLI's own session log, parsed by [evidence/session-report.py](#). Reported separately from rubric 2e, which concerns the runtime of the compiled program and is unrelated to these numbers.

Run 00 — generation prompt only

Both systems were asked for the program, then asked separately for the report. The figures below cover **only the generation prompt**, so the two are comparable; the report prompt is excluded. Cache reads are excluded from the work figure because they are text the provider had already processed.

	Claude	Codex
Model	claude-opus-5	gpt-5.6-sol, effort xhigh
Harness	Claude Code	Codex Desktop 0.153.4 (VS Code)
Wall clock	1 m 59 s	1 m 41 s
API requests / responses	7	6
Output tokens	10,282	4,874
of which reasoning	(not itemised)	1,454
Fresh, non-cache-read tokens	39,183	19,201
Cache reads (excluded above)	421,446	156,544
Program produced	382 lines, 10,141 B	293 lines, 6,780 B
Compiled and tested itself?	yes	yes
Disclosed rule violation?	yes, unprompted	yes, unprompted

Claude spent roughly twice the fresh tokens and produced roughly 1.5× the code in about 18 seconds more wall clock. Whether the extra output bought anything is a question for 2e, not for this table — length is not quality, and the correctness bench has not been run yet.

A caution on the Codex figure: its own report put the generation prompt at 1 minute 46.157 seconds from task metadata, while the rollout log's first and last timestamps for that turn give 101 seconds. The two clocks start at slightly different moments. The log figure is used for both systems for consistency, and the discrepancy is recorded rather than averaged away.

Space complexity

Static code analysis. Each program keeps one array of email records that grows as emails arrive, plus three separate pieces of text per email for the sender, subject and date. Both double the array whenever it fills. Memory therefore grows in direct proportion to the number of emails — $O(n)$ for both.

The one static difference: Codex stores each email's date in a type that takes 4 bytes and Claude uses one that takes 8, so a Codex email record is 40 bytes against Claude's 48. Claude also starts its array at 16 slots and Codex at 8.

Dynamic profiling (information generated in each Als report by Valgrind):

Emails (n)	Claude	Codex	Difference
100	19,325 B	18,301 B	5%
1,000	102,805 B	94,613 B	8%
10,000	1,249,784 B	1,122,997 B	11%

Ten times the emails costs roughly ten times the memory at both steps, so the measured curve is **linear**, exactly as static analysis predicted, and neither program is quadratic. Per email it settles at about 125

bytes for Claude and 112 for Codex.

Codex is genuinely leaner, and the reason is the 8 bytes per email from the narrower date type. This is the one place in the entire comparison where a real, repeatable difference shows up — and it is worth about 127 KB at ten thousand emails.

Valgrind also reports zero errors and zero leaked memory for both programs. Every piece of memory either one asks for is given back.

Maintainability

Comments.

	Claude	Codex
Comment lines	53	4
Comments per function	2.3	0.3

Claude's comments are correct where they exist and explain why a piece of code is there. Codex's four are section markers and explain nothing.

Neither file has a usable prolog. The rubric lists nine things that belong at the top — program name, description, inputs, outputs, author, creation date, revision date, collaborators, and other sources including GenAI. I checked both against all nine. Claude has two. Codex has none; its first line is `#include <stdio.h>`.

Neither model wrote my name and neither said a GenAI wrote the code.

Ccontextual readability. Claude gives each of the four commands its own named function, so changing what `NEXT` prints means editing one short function. Codex handles all four inside `main`, which reads well top to bottom but means `main` does several jobs at once. Naming is good in both; Codex's is slightly more verbose (`insertion_order` against `seq`).

Edge case and error handling. Both models did more here than the lecture's example expects — both check every memory request, both handle a missing file, neither crashes. The gap is the two silent failures already listed under Correctness.

Automated static analysis — cyclomatic complexity. This counts the decision points in a function; more branches means more ways to get it wrong when you change it. The lecture says to flag anything above 15.

	Claude	Codex
Highest function	8 (<code>readLine</code>)	11 (<code>main</code>)
Functions above 15	none	none

Both pass. Claude's highest is lower because it split the work into more functions; Codex's `main` carries the command dispatch that Claude moved out.

Style compliance. Both follow a consistent style throughout. Codex produces one warning under the oldest C standard where Claude produces none; both are clean under the standards the grading environment uses, so this changes nothing practical.

2f. Which one was selected, and why

I am building on Codex's version.

The reasoning is not that Codex scored better. Claude won more of the eight comparisons. The reasoning is that almost everything Claude won is a thing I can simply *instruct*, and the one thing Codex won is not.

Sort the differences by whether a rule in **AGENT.md** can erase them:

Difference	Winner	Can a written rule fix it?
Comment lines, 53 against 4	Claude	Yes — "comment every line"
No usable prolog header (both files)	neither	Yes — list the nine items
Four commands split into four functions	Claude	Yes — "one function per command"
Silent when the inbox is empty	Claude	Yes — "print a message"
Silently ignores extra arguments	Codex	Yes — "reject them"
Tie broken by first-in instead of last-in	—	Yes — "first in"
Date type overflows on an out-of-range year	Claude	Yes — one keyword
Uses 11% less memory for the same job	Codex	No

Every row but the last is a sentence I can write down. The last one is a design decision made while the program was being built: a smaller record for each email, a smaller starting array, less of everything for the same result. I can tell a model to comment its code and it will. Telling a model to "be economical" does not reliably produce a leaner data structure.

So choosing on comment count would mean choosing on the most fixable axis available, which is backwards. The right question is which file's *judgment* I want underneath, and the measurement that reveals judgment is the one Codex won.

2g. Improvement

I rewrote AGENT.md and reran the Codex agent in a new Codex chat with the same prompt as before.

Results — did the rules take?

Yes, on every instructable item. I regenerated with the new **AGENT.md** in place and saved it as **iterations/01-raw.c**.

Rule	00-raw	01-raw
Prolog, the nine rubric items	0 of 9	9 of 9
Comment lines	4	261 , out of 288 lines

Rule	00-raw	01-raw
NEXT on an empty inbox	printed nothing	No emails to read.
Tie on category and date	first in the file	first in the file, kept
Date key type	int, overflows	long, sanitizer reports nothing
Extra command-line argument	usage message, exit 1	kept
Complexity of main	11	8
Functions	14	16, one per command

Nothing broke on the way. 01-raw still reproduces the sample byte for byte on both input paths, still orders all 500 random emails correctly, still reports zero errors and zero leaked memory under Valgrind, and still compiles silently under `-Wall -Wextra -O2`. Eight rules written down, eight rules applied, no regressions in correctness.

But it cost me the reason I chose this file.

Peak memory at 10,000 emails:

	Peak heap
Codex 00-raw	1,223,988 B
Codex 01-raw	1,359,678 B
Claude 00-raw	1,364,167 B

The whole 11% is gone. 01-raw now sits within 0.3% of the Claude program I rejected.

I can point at exactly where it went. Widening the date key from `int` to `long` adds 8 bytes to every email record. At 10,000 emails the heap's array holds 16,384 slots, and $16,384 \times 8 = 131,072$ bytes, which is the gap I measured.

So the 11% advantage and the overflow bug were not two facts about Codex's program. They were one fact. Codex was leaner because it used a type too narrow to hold the number it was storing, and the moment I fixed that, the program became the size Claude had chosen on its own at run 00, without being told.

In 2f I argued that the memory result was the one difference a written rule could not reproduce, and that this made it evidence of design judgment. That was wrong, and it was wrong in a way I could have predicted — in the same section I wrote that the leanness and the defect were one decision seen from two sides, and then treated them as separable anyway. The keyword that fixed the bug was the keyword that bought the advantage.

One thing did improve that I did not ask for. 01-raw is now the fastest of the three at 10,000 emails — 0.692 s against 0.730 for Codex 00 and 0.789 for Claude, best of five runs. That is about 5%, which is the same size as the gap I dismissed as noise earlier in this report, so I am recording it and not claiming it.

Correctness. I settled the two places the models disagreed — an empty inbox now prints `No emails to read.` instead of nothing, and a tie on both sender category and date is broken in favour of whichever

email came first in the file — and I widened the date key from `int` to `long` so a large year can no longer overflow it. All three are verified: the sample still matches byte for byte, all 500 randomly generated emails still come out in the right order, and the overflow sanitizer that flagged 00-raw now reports nothing.

Execution time. I changed nothing, deliberately. The benchmarks put both 00-raw programs on the same growth curve, so any optimisation would have been decoration bought at the risk of breaking working code. It came out about 5% faster anyway as a side effect of splitting the commands into separate functions, which I am recording rather than claiming, because 5% is the same gap I called noise earlier in this report.

Space complexity. I made it worse on purpose. Widening the date key added 8 bytes per email record, which is 131,072 bytes at 10,000 emails and gave back the entire memory advantage this program was chosen for. Correctness outranks 128 KB on a test file of a dozen commands, so I took the trade knowingly.

Maintainability. This is where nearly all the improvement is. The prolog went from none of the nine required items to all nine, the comment count went from 4 lines to 261 out of 288, each of the four commands moved into its own named function, and the complexity score of `main` dropped from 11 to 8 because it now reads lines and hands off instead of doing the work itself.